

Funcionamiento

Al ejecutar el programa, lo primero que se hace es pedirle al usuario la palabra que desea buscar en la sopa de letras previamente suministrada, la cual debe ser 50x50. Esto se hace mediante “*System Services*”, los cuales almacenan la palabra ingresada en un espacio reservado en memoria para las palabras que ingrese el usuario. Luego de esto, se siguen una serie de pasos generales para la búsqueda de la palabra:

Step 1: En este paso se realiza lo siguiente:

1. Se guardan la primera letra de la sopa de letras y la primera letra de la palabra a buscar en un par de registros. Con uno recorreremos la sopa y con otro la palabra a buscar.
2. Mediante un ciclo “*For*”, recorreremos la sopa actualizando el registro destinado para ello, y se compara con el registro que almacena la primera letra de la palabra hasta que se encuentre que sean iguales. En este momento se pasa al *Step 2*. Por el contrario, si el ciclo acaba y no se encuentra una coincidencia, se le informará al usuario que la palabra no se encuentra en la sopa de letras y se le preguntará si desea terminar la ejecución del programa o buscar otra palabra.

```
word = [c,a,s,a]; // Ingresado por el usuario
soup = // Array con todas las letras y caracteres de la sopa
actual_word = word[0];
actual_soup = soup[0];

for (i = 0, i < soup.length, i++) {
    if (actual_word == actual_soup) {
        step_2();
    } else if (actual_soup == "\n" or "\r") {
        continue;
    }
    actual_soup = actual_soup[i]
}
```

Pseudocódigo que muestra el funcionamiento del *Step 1*.

Step 2: Luego de haberse completado el paso 1 encontrando una coincidencia, realizamos lo siguiente:

1. Ya que la primera letra de la palabra que buscamos fue encontrada, guardamos la segunda letra en un registro para comprobar la dirección hacia la que podría estar la palabra.
2. Comparamos la segunda letra de la palabra buscada con las letras que se encuentran justo arriba, abajo, a la izquierda y a la derecha de la letra en que nos detuvimos en el *Step 1*. Si en algún caso son iguales, se continúa con el *Step 3*, de lo contrario regresamos al *Step 1* y continuamos con el recorrido.

```
Step_2() {
    actual_word = actual_word[1];
    left = // Letra a la izquierda de "actual_soup"
    right = // Letra a la derecha de "actual_soup"
    up = // Letra arriba de "actual_soup"
    down = // Letra debajo de "actual_soup"
    if (actual_word == left) {
        Step_3(left);
    } else if (actual_word == right) {
        Step_3(right);
    } else if (actual_word == up) {
        Step_3(up);
    } else if (actual_word == down) {
        Step_3(down);
    } else {
        actual_word = actual_word[0]
        return;
    }
}
```

Pseudocódigo que muestra el funcionamiento del *Step 2*.

Step 3: Teniendo en cuenta la dirección hacia la que encontramos la segunda letra de la palabra, saltamos a la función de la dirección respectiva en la cual se realiza lo siguiente:

1. Se comparan el resto de letras de la palabra con las letras en la dirección elegida en el *Step 2* hasta que hallemos que todas las letras de la palabra coinciden con las de la sopa o hasta que haya una interrupción.
2. Si todas las letras fueron encontradas (se encontró la palabra), se salta a la función respectiva en el *Step 4*. De lo contrario, se regresa al *Step 2* para

terminar de confirmar el resto de direcciones en caso de que no se hayan validado todas.

3. En el caso de que en ninguna de las direcciones se haya encontrado la palabra completa, se regresa al *Step 1* para continuar con el recorrido.

```
Step_3(direccion) {
    switch(direccion) {
        case left:
            letter = // Siguiente letra a la izquierda
            for (i = 2, i < word.length, i++) {
                if (letter == word[i]) {
                    continue;
                } else {
                    return;
                }
            }
            Step_4(left);
            break;
        case right:
            letter = // Siguiente letra a la derecha
            for (i = 2, i < word.length, i++) {
                if (letter == word[i]) {
                    continue;
                } else {
                    return;
                }
            }
            Step_4(right);
            break;
        // Lo mismo con las otras 2 direcciones
    }
}
```

Pseudocódigo que muestra el funcionamiento del *Step 3*.

Step 4: En caso de haber encontrado la palabra en el *Step 3*, saltamos a la función respectiva del *Step 4*, la cual realiza lo siguiente:

1. Se calcula la posición en la que se encuentra la primera letra de la palabra buscada mediante una función llamada “Pos”.
2. Mediante “System Services” se imprimen el número de fila, número de columna y dirección en la que se debe leer la palabra que el usuario buscaba.
3. Se salta a la función “Continue”, la cual también mediante un “System Service” le pregunta al usuario si desea seguir buscando palabras. En caso de que diga que no, se termina la ejecución del programa.

```

Step_4(direccion) {
    row = position.row()
    column = position.column()
    print("La palabra está en la fila" + row)
    print("La palabra está en la columna" + column)
    switch (direccion) {
        case left:
            print("La palabra se lee de derecha a izquierda")
            return;
        case right:
            print("La palabra se lee de izquierda a derecha")
            return;
        case up:
            print("La palabra se lee de abajo a arriba")
            return;
        case down:
            print("La palabra se lee de arriba a abajo")
            return;
    }
}

```

Pseudocódigo que muestra el funcionamiento del *Step 4*.

Decisiones de Diseño

A continuación se nombran una serie de problemas o situaciones que fueron encontrados mientras se realizaba el código y la forma en que fueron solucionados:

1. **Finales de Recorridos:** Lo primero que se notó, fue que era necesario reconocer cuándo se terminaban de recorrer tanto la sopa de letras como la palabra buscada. Para esto se reservó un espacio en memoria llamado “end_file”, en el cual se guardó un 0. Con este indicador, se sabría que si durante el recorrido de la sopa nos encontrábamos con un 0, el recorrido había terminado. En cuanto a la palabra se notó que el “*System Service*” número 54 automáticamente agregaba un salto de línea (\n) al final del String ingresado por el usuario, por lo que se decidió utilizar el mismo como indicador del final de la palabra.
2. **Registros para Cada Función:** Una decisión de diseño, para mantener un estándar y facilitar la escritura del código, fué utilizar los registros 0 (\$t0, \$a0, \$s0) como auxiliares, los registros 1 para letras y direcciones de la sopa de letras y los registros 2 para letras y direcciones de la palabra que se desea buscar.

- 3. Impedimentos al Usuario:** Para la correcta ejecución del programa, se debían tener contramedidas para las acciones que hiciera el usuario, fueran por error o intencionadas. El primer caso se da al ingresar la palabra que desea buscar, ya que si el usuario no ingresa ninguna palabra, o si selecciona la opción “*Cancel*”, se le enviará una alerta diciéndole que el espacio está vacío o preguntándole si desea terminar la ejecución del programa, respectivamente. Otro caso que se tuvo en cuenta era si el usuario intentaba oprimir el botón “*Cancel*” cuando se le preguntaba si desea seguir buscando palabras o no, para lo cual se le muestra un mensaje que le dice que sólo puede escoger SI o NO.
- 4. Alineamientos al Recorrer la Sopa:** Debido a que se decidió trabajar con el archivo de texto con la sopa de letras tal y como fue entregado, es decir, sin eliminar los espacios ni las secuencias de escape (\n y \r), el recorrido se hacía saltando de a 2 espacios de memoria para así omitir los espacios. Sin embargo, llega un punto, precisamente en los saltos de una línea a otra, en el que nos encontramos inevitablemente con un \n, por lo que se creó la función “*Align*”, cuyo único objetivo es corregir este caso sumando 1 al contador que recorre la sopa y luego continuando con la ejecución normal.
- 5. Retornos:** Algo muy importante a tener en cuenta fueron los retornos o “*CheckPoints*”, los cuales nos sirvieron para regresar a ciertos puntos de ejecución del programa en caso de haber entrado a una función para comprobar una dirección y que al final no se encontrara el resultado esperado. Los checkpoints utilizados fueron:

 - a. *Resume*: Este checkpoint se utiliza en los Steps 1 y 2. Se regresa a él en caso de que comprobando en todas las direcciones válidas no se hayan encontrado la totalidad de las letras de la palabra buscada, por lo que se continúa con el recorrido de la sopa.
 - b. *KRight*, *KLeft*, *KUp*: Estos checkpoints se encuentran en el *Step 2* y cumplen la misma función: Volver a la línea que está justo después de la línea que nos envía a la búsqueda de la palabra en dichas direcciones del *Step 3*. Se regresa a ellos en caso de que en sus respectivas funciones de búsqueda en esa dirección no se haya

encontrado la totalidad de la palabra. Son necesarios debido a que la comprobación de direcciones se realiza en este orden: Derecha, Izquierda, Arriba, Abajo; por lo que si en la primera dirección no se encontró la palabra, aún faltaría comprobar las demás direcciones. Debido a esto es que no existe un *KDown*, ya que es la última dirección por comprobar, y en caso de no estar ahí la palabra, directamente se regresa al checkpoint *Resume*.

6. **Direcciones Inválidas:** Se tuvieron en cuenta 2 direcciones inválidas al momento de buscar la palabra. Una dirección es hacia arriba en caso de estar recorriendo la primera fila de la sopa, ya que arriba no hay nada; y la otra dirección es hacia abajo en caso de encontrarnos recorriendo la última fila de la sopa. Esta decisión fue tomada principalmente para ahorrar ejecución de código. Se puede observar su implementación en las líneas de código 87-89, 93-95.
7. **Guardar Valores en Pila:** Como complemento del numeral 4 (Retornos), se notó que debíamos guardar en la pila el contenido de los registros que íbamos a modificar en el *Step 3*, ya que si llegáramos a necesitar volver a un checkpoint, no nos servirían los valores de los registros modificados, sino que necesitamos los valores originales que llegaron al *Step 3*. Para esto se crearon las funciones “*Store*” y “*Load*”, las cuales guardan los valores de los registros calculados en el *Step 2*, y los vuelven a cargar de ser necesario, respectivamente.
8. **Código Repetido:** En muchas ocasiones se notó que muchas partes del código se repetían exactamente igual y hacían lo mismo, por lo que se decidió crear funciones con estos fragmentos de código y llamarlas en los momentos que fuera necesario, para así ahorrar líneas en el programa. Algunos de estos casos son fueron:
 - a. *Store* y *Load*: Su funcionamiento fue especificado anteriormente.
 - b. *Next*: Calcula y guarda la siguiente letra de la palabra buscada.
 - c. *Pos*: Calcula mediante el contador que recorre la sopa, la fila y columna en que se encuentra la palabra.

d. *Result*: Imprime el resultado calculado en *Pos*.

9. Calcular Fila y Columna: Luego de encontrar la palabra buscada, el cálculo de la fila y la columna en que se encontraba la primera letra de la palabra se realizó mediante una serie de operaciones aritméticas. Primero, se dividió el contador con el que recorrimos la sopa por 101 (número de caracteres por fila). Con esto, se guardaría en “Lo” y “Hi” el cociente y el residuo de la operación, los cuales equivalen a la fila y la columna en que se encuentra la letra, respectivamente. Algo a tener en cuenta fué que el número de columna se daba teniendo en cuenta los espacios en el archivo, por lo que dividimos este número entre 2 (Así obtendremos el número de espacios), y luego al número original le restamos este resultado, dando así la columna correcta.

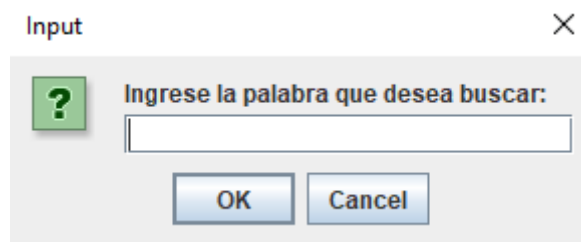
Ejecución

Lo primero que debe hacer el usuario al abrir el código, y antes de ejecutarlo es especificar la ruta del archivo que contiene la sopa de letras en el espacio “*file_in*” (tercera línea del código), utilizando 2 barras invertidas para separar cada carpeta o directorio.

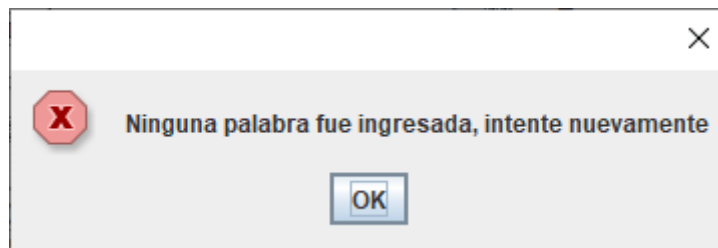
```
"D:\\Universidad\\Arquitectura de Computadores y Laboratorio\\Laboratorio 2\\wor
```

Luego de esto, el usuario podrá ejecutar el código:

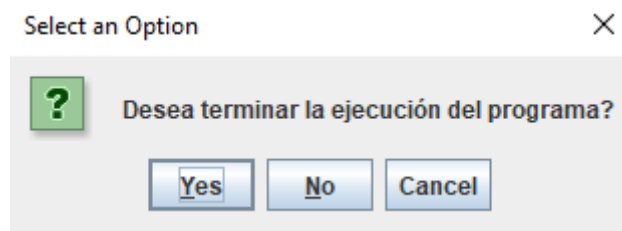
Lo primero que se encontrará al ejecutar será una ventana emergente en la cual deberá ingresar la palabra que desea buscar. La ventana tendrá 2 botones: “Ok” y “Cancel”. Dependiendo de lo que realice el usuario el código continuará su ejecución con normalidad o se ejecutarán ciertas excepciones.



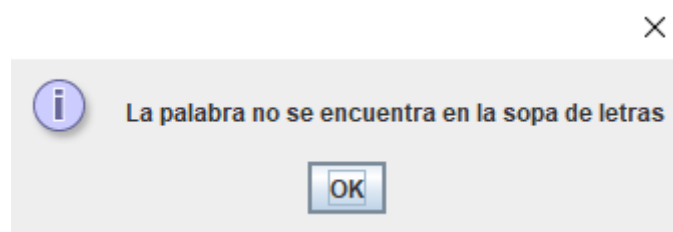
En caso de que el usuario oprima “Ok” sin ingresar ninguna palabra, saldrá un mensaje de error y regresará al recuadro anterior.



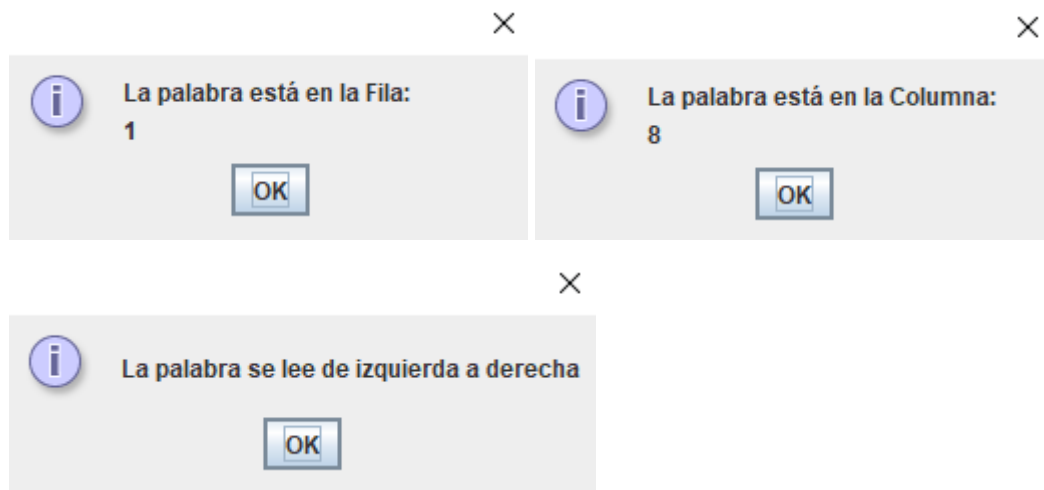
En caso de que seleccione “Cancel”, se le preguntará si desea terminar con la ejecución del código o no.



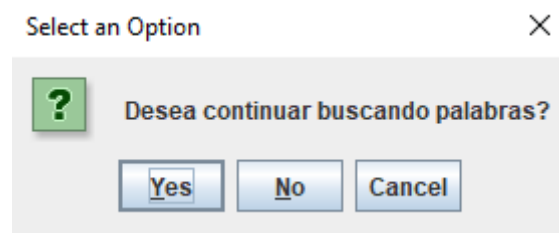
Suponiendo que el usuario ingresa una palabra adecuada, pero esta no se encuentra en la sopa de letras, se le mostrará una ventana con el siguiente mensaje.



Ahora, en caso de que la palabra haya sido encontrada, saldrán 3 ventanas emergentes, una por una especificando la fila, columna y dirección en que se lee la palabra buscada.



Luego de esto, se le preguntará al usuario si desea seguir buscando palabras o si desea terminar con la ejecución del programa.



A continuación adjuntamos el link de la sustentación del código fuente que ejecuta el programa: (Tratamos de resumir pero era muy largo el código)

[Enlace al Video Explicativo](#)